HAL open science

Ariel OS: An Embedded Rust Operating System for Networked Sensors & Multi-Core Microcontrollers

Elena Frank, Kaspar Schleiser, Romain Fouquet, Koen Zandberg, Christian Amsüss, Emmanuel Baccelli

To cite this version:

Elena Frank, Kaspar Schleiser, Romain Fouquet, Koen Zandberg, Christian Amsüss, et al.. Ariel OS: An Embedded Rust Operating System for Networked Sensors & Multi-Core Microcontrollers. DCOSS-IoT 2025 - IEEE International Conference on Distributed Computing in Smart Systems and the Internet of Things, Jun 2025, Lucca, Italy. hal-05119288

HAL Id: hal-05119288

https://inria.hal.science/hal-05119288v1

Submitted on 18 Jun 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

Ariel OS: An Embedded Rust Operating System for Networked Sensors & Multi-Core Microcontrollers

E. Frank[2], K. Schleiser[1,2], R. Fouquet[1], K. Zandberg[2], C. Amsüss[1,3], and E. Baccelli[1,2,3]

[1]Inria, France
[2]Freie Universität Berlin, Germany
[3]Einstein Center Digital Future, Germany

***Abstract*—Large swaths of low-level system software building blocks originally implemented in C/C++ are currently being swapped for equivalent rewrites in Rust, a relatively more secure and dependable programming language. So far, however, no embedded OS in Rust supports multicore preemptive scheduling on microcontrollers. In this paper, we thus fill this gap with a new operating system: Ariel OS. We describe its design, we provide the source code of its implementation, and we perform micro-benchmarks on the main 32-bit microcontroller architectures: ARM Cortex-M, RISC-V and Espressif Xtensa. We show how our scheduler takes advantage of several cores, while incurring only small overhead on single-core hardware. As such, Ariel OS provides a convenient embedded software platform for small networked devices, for both research and industry practitioners.**

***Index Terms*—Embedded Software, Rust, Microcontroller, Multicore, Operating System, RTOS, Internet of Things, IoT**

I. Introduction

We increasingly depend on cyberphysical systems and distributed computing systems which encompass not only machines in the microprocessor segment, but also much more resource-constrained devices such as sensors based on microcontroller units (MCU). The latter, as characterized in RFC7228 [1], achieve ultra-low power consumption and ultra-low price tags, but provide much less memory (in the *kilobyte* range) and much weaker processing power (CPU clock speeds in the *megahertz* range) compared to microprocessors.

A flurry of multicore microcontrollers — Exploiting multicore efficiently is required to allow the execution of computation-intensive tasks, such as real-time audio processing or edge machine learning, while the device remains available for sensing, actuation, or push/pull of data over the network. Thus, many vendors' flagships are now based on multicore 32-bit architectures. Examples include the Espressif ESP32-S3 microcontrollers based on a dual-core Xtensa LX7; the RP2350 microcontrollers based on dual-core RISC-V Hazard3 and dual-core ARM Cortex-M33, used on popular boards such as the Raspberry Pi Pico 2. An earlier model, the RP2040, based on dual-core ARM Cortex-M0+, had already sold millions of units. Other vendors (Nordic, NXP, ST, etc.) also rolled out 32-bit multicore microcontrollers.

Embedded operating systems for microcontrollers — As software running MCUs and sensors became more complex, the use of an operating system (OS) became frequent. The most prominent operating systems are written in C (RIOT, Zephyr, FreeRTOS [2]). Recently, a new breed of OS and embedded software platforms has emerged, written in Rust.

This effort was pioneered by Tock OS [3] and many *bare-metal* embedded Rust programming efforts, which resulted in vastly improved embedded Rust [4]. Examples of embedded Rust platforms for MCUs include Hubris [5], Drone OS [6], RTIC [7], as well as asynchronous Rust with Embassy [8]. However, to date, none of these support multicore scheduling on the main 32-bit microcontrollers.

Aiming to bridge this gap:

- We design Ariel OS, a novel embedded Rust operating system combining (i) a scheduler exploiting single- and multicore microcontrollers, and (ii) asynchronous Rust;
- We implement Ariel OS and provide benchmarks on diverse 32-bit microcontroller architectures: ARM Cortex-M, Espressif Xtensa, and RISC-V;
- We overview the OS, which, beyond the scheduler, integrates diverse libraries and cross-hardware APIs;
- We publish the full Ariel OS code as open source.

II. Prior work on Multicore Scheduling on MCUs

Surveying existing software and prior literature, we found that notable examples of multicore scheduling on microcontrollers include ThreadX [9] and FreeRTOS [10], Zephyr [11] and NuttX [12]. Typically, threads that are ready and waiting for execution are listed in a sorted data structure called the *runqueue*. We observe two main approaches for runqueues on multicore architectures. The first approach, *global scheduling*, uses a single global runqueue from which the threads are distributed onto the available cores. In contrast, the *partitioned scheduling* approach employs separate runqueues for each processor, to which threads are statically allocated. Furthermore, we observed that OSs for microcontrollers that support multicore scheduling primarily use global scheduling. We also observed that for synchronization within the scheduler, global critical sections are typically used.

We finally observed that operating systems in this space employ different approaches for assigning threads to cores. On the one hand ThreadX uses a *core reallocation approach*, whereby a reallocation routine maps the n highest priority threads to the N cores. The allocation for each core is read

by the scheduler interrupt handler upon invocation. After each change in the runqueue, the reallocation routine—in ThreadX called *rebalance*—is executed.

On the other hand, FreeRTOS, Zephyr or NuttX use variants of a *dynamic thread selection approach*, whereby the next thread for a core is selected directly from the runqueue by the scheduler interrupt handler upon invocation. The thread is then removed from the runqueue to prevent it from being selected for multiple cores. Conversely, when a running thread is preempted, it is re-added to the runqueue.

III. Goals of Ariel OS and its Scheduler

Use-Case & Assumptions — Our work targets typical microcontrollers, which entails N cores and n threads, and only shared caches, if any. Furthermore N and n are small: typically $N < 3$ and $n < 15$. Notice major differences versus Linux, L4 or HPC use-cases, which entails much larger N and n, more elaborate cache and fairness mechanisms.

In this paper, we aim to support multicore, for Symmetric Multiprocessing (SMP) cases, which is the most common on microcontrollers so far, whereby the multiple cores are identical in their architecture and share the entire address space. We pursue the following objectives:

- **Work-Conservation —** the scheduler must execute the *n* highest priority threads on the *N* available cores;
- **Real-Time —** Retain the real-time properties of the scheduler, based on thread priorities set by the user;
- **Portability —** Ensure portability between heterogeneous architectures and platforms.
- **Transparency —** Transitioning from single-core to multicore use must be transparent, retain scheduler performance, and not increase complexity for the user.

Operating System — Beyond scheduling, Ariel OS aims to be a one-stop-shop for distributed computing and networked applications on heterogeneous 32-bit MCUs. More details on this bigger picture are given in Section VIII.

IV. Single-Core Scheduler Basis & Optimization

The Ariel OS builds upon a single-core scheduler basis similar to RIOT [13]. That is: a tickless real-time scheduler with a preemptive priority scheduling policy. The maximum number of threads is set at compile time, and all scheduler data structures are allocated statically. Context switching is done lazily, in that a context is only switched when the head of the runqueue has changed since the last scheduler invocation. The scheduler is further optimized to avoid superfluous scheduler interrupts. Specifically, Ariel OS checks whether a new thread is ready and has higher priority than the current one *before* setting a scheduler interrupt.

V. Ariel OS Multicore Scheduler Design

In the following, without loss of generality, we assume $N = 2$ cores (Core 0 and Core 1). Note, however, that scaling to $N > 2$ is trivial, assuming N and n remain small as per our goals (recall section III).

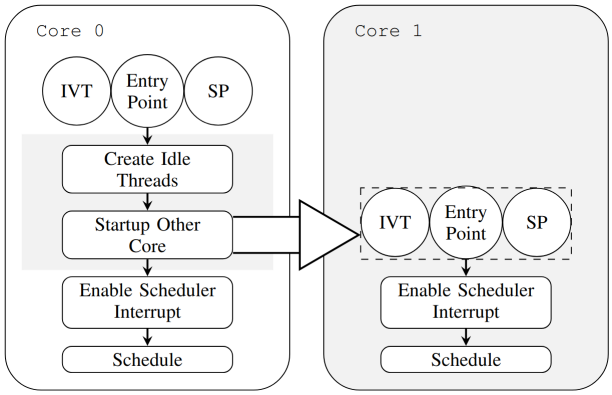


Fig. 1: Threading startup on a dual-core system.

A. Startup

The Ariel OS runtime initialization executes on Core 0, while the other core is in deep sleep or stall mode. After the general initialization on Core 0, Core 1 is set up with the shared interrupt vector table (IVT), a separate main stack pointer (SP), and an entry function that, down the line, will start threading on this core, analogous to Core 0. This is depicted in Fig. 1. Threading is started by enabling the scheduler interrupt at the lowest priority and invoking the scheduler, which will then select the next thread for the core.

B. Global Scheduling Scheme

Ariel OS uses a global scheduling scheme. Global scheduling facilitates thread migration, where threads migrate between cores between executions. We chose global scheduling because it reduces context switches and priority inversions, and makes better use of the overall capacity [14], [15]. Furthermore, it avoids the thread allocation problem of partitioned scheduling, i.e., the problem of finding an optimal allocation of threads to cores. The main drawback of global scheduling is scalability, when the global runqueue becomes a bottleneck for context switching [16]. However, this issue is negligible on MCUs — which have a low processor count.

C. Assigning Threads to Cores

With the single-core configuration, the thread-selection process simply reads the head of the runqueue. However, on multicore, this would result in the same thread being executed on multiple cores. There are various approaches for selecting the next thread that a core should execute. We initially designed our scheduler so that it can accommodate either *core reallocation* or alternatively *dynamic thread selection* (see section II). We then implemented both, before reporting on their performance on different hardware in section VII.

D. Core Affinity Masks

Core affinities are a feature that allows pinning a thread so it runs on some specified core(s) only. This provides the user with fine-grained control over how and where threads may run. Core affinities are realized with a bitmask, where a

set bit encodes that the thread can run on the core with this index. The scheduler uses this information when selecting the core on which a thread should run.

E. Dynamic Priorities and Priority Inheritance

Thread priorities in Ariel OS can be changed dynamically at runtime for all active threads. A change in priority may result in a context switch if a running thread's priority is decreased or if a waiting thread's one increased. Furthermore, dynamic priorities facilitate priority inheritance for mutexes. If a higher-priority thread is blocked on a mutex that is currently owned by a lower-priority thread, the owning thread inherits the higher priority of the waiting thread. This prevents the owner from being preempted by other lower-priority threads and thus helps avoid priority inversions caused by indirect blocking.

F. Synchronization in the Scheduler

Ariel OS implements mutual exclusion in the kernel through a global critical section, effectively resulting in a *Big Lock* design. Thus, there is no concurrency in the kernel, which is tolerable given that such operations in Ariel OS are short. It ensures correctness, prevents data races and deadlocks, and simplifies operations that involve multiple data structures, such as the runqueue and Thread Control Blocks (TCBs), because no individual locking is needed. On single-core systems, it is sufficient to mask all interrupts to create a critical section. In multicore systems, an additional global spinlock is required to ensure that even threads on other cores cannot enter another critical section.

G. Combining Scheduled Threads & Async Rust Tasks

Ariel OS uses async code based on *Embassy* [8] for system initialization and in the HAL. There is always a system executor for async tasks. Using the preemptive scheduler — and thus threading — is optional. If the scheduler is not used, the system executor will run in interrupt context. If the scheduler is used, the system executor executes in a thread. Additional executors can be started in other threads. When all tasks on an executor are pending, the owning thread is suspended. Threads can block on async functions and wait for async resources from an executor. Thus, the gap is bridged between the scheduler, async Rust, future-based concurrency, and asynchronous I/O.

VI. Ariel OS Scheduler Implementation

As of this writing, Ariel OS supports multicore scheduling on three popular hardware platforms/architectures: the RP2040 and RP2350 (dual-core ARM Cortex-M0+/ dual-core ARM Cortex-M33 resp.) and the ESP32-S3 (dual-core Xtensa LX7). RISC-V multicore scheduling support in Ariel OS is currently a work in progress.

The RP2040, RP2350, and the ESP32-S3 are well supported in the Rust ecosystem: the former through the Embedded-Rust working group and the `rp-rs` project, the latter directly by Espressif through the `esp-hal` project. Ariel OS leverages this support in its implementation.

A. Hardware Abstraction

One of the main objectives of our work is clear hardware abstraction and the reduction of platform-specific code. Adding support for another chip should be minimal effort, particularly when there is already chip support in the ecosystem. In Ariel OS, hardware abstraction occurs at two levels:

CPU architecture abstraction — At this layer, the scheduler logic for a CPU architecture, e.g., Cortex-M, is implemented. It involves all architecture-specific code to set up a thread stack, configure the exception used to trigger the scheduler, and the actual context-switching logic.

Chip-level abstraction — The platform-specific logic for SMP is implemented at this layer. For multicore scheduling, two mechanisms are required, (i) for starting up the other core(s), and (ii) for invoking the scheduler on a specific core.

Ariel OS takes advantage of the Rust type system, by specifying the above abstractions as *traits* [17]. Adding support for another platform in the scheduler only requires two traits – one per layer – to be implemented. As a result, the corresponding chip-level SMP implementations in Ariel OS are only 70 lines of Code (LOC) for the RP2040 and 66 LOC for the ESP32-S3.

B. Platform-specific Scheduling Logic

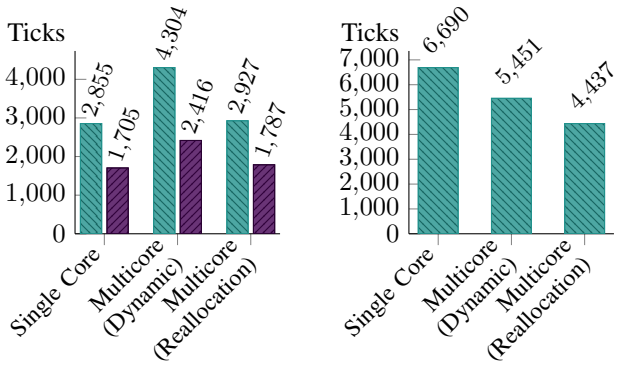
On the RP2040/RP2350 — The chip implements (in hardware) two FIFO queues between the physical cores, which are utilized by Ariel OS during startup and for scheduler invocations. During startup, Core 1 remains in sleep mode until it receives the vector table, stack pointer, and entry function through the FIFO queue, based on a fixed protocol. After startup, the FIFO queue is used to invoke the scheduler on the other core. A received message will trigger an interrupt, where the handler will set the local scheduler exception.

On the ESP32-S3 — The chip does not implement any inter-processor communication channel. Instead, two different software-triggered CPU interrupts are used to invoke a scheduler on each core, respectively. Startup of the second core on the ESP32-S3 is implemented by writing the address of the entry function into the boot address of Core 1, then resetting and unstalling the core. The entry function sets up the vector table address and the stack pointer for this core, and then runs our threading startup logic.

C. Interrupt Handling

On the RP2040/RP2350, each core is equipped with its own ARM Nested Vectored Interrupt Controller (NVIC). On the ESP32-S3, each core has its own configurable interrupt matrix. On both chips, nested interrupts are supported, so that a lower-priority interrupt can be preempted by a higher-priority one. External interrupts are routed to both cores, but only enabled on one core for handling, as described below.

During startup, peripherals are initialized either in interrupt mode on Core 0 before threading is started, or by one high-priority thread. This initialization configures and enables the required interrupts on the core it executes on, which will then be the core that handles the related interrupts.



(a) RP2040/RP2350 (blue/purple) (b) ESP32-S3 (dual Xtensa LX7)

Fig. 2: Context switching performance of the two multicore scheduler designs compared with single-core configuration.

Still, both cores share the same interrupt vector table, so it is also possible to manually mask and unmask interrupts on individual cores to configure where an interrupt should be handled.

D. Synchronization in the Scheduler

The scheduler is implemented through a single structure that contains the runqueue, TCBs, and other scheduling data, and implements all scheduling logic. This structure is protected by a wrapper type that ensures that all accesses to the scheduler are executed inside a critical section and that no reference to the scheduler can be obtained outside of it.

VII. Micro-Benchmarks with Ariel OS Multicore

We next evaluate the performance of Ariel OS on popular hardware. We published our benchmark code in [18].

The **dual-core MCUs** we used are: (i) Espressif ESP32-S3 with dual Xtensa LX7 at 240 MHz, and (ii) RP2040/RP2350 with dual Cortex-M0+/M33 at 133/150 MHz.

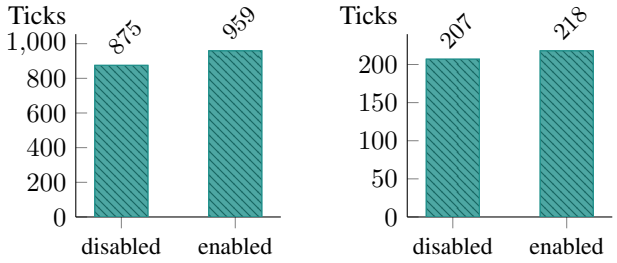
The **single-core boards** we used for comparison are: (i) a Nordic nRF52840 with a Cortex-M4 at 64 MHz, and (ii) an Espressif ESP32-C3 with a RISC-V RV32IMC at 160 MHz.

On Xtensa and Cortex-M, the performance is measured in ticks. On the RISC-V-based ESP32-C3, we instead use the system timer, which runs at 16 MHz. The measurements we present are the average of 1000 runs.

A. Comparing Multicore Scheduler Variants

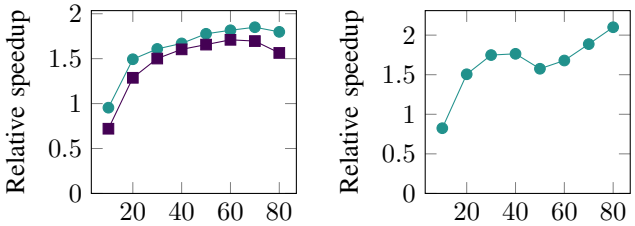
Figure 2 depicts micro-benchmarks in which four threads, grouped into pairs, alternate in waking each other up and then suspending their own execution. We compare the performance of different variants of the scheduler: (i) the *single-core* variant described in section IV, (ii) a multicore variant using *core reallocation*, and (iii) a multicore variant using *dynamic thread selection* as mentioned in subsection V-C.

The results show that *dynamic* outperforms *reallocation* on all the multicore microcontrollers we tested. The relatively more complex *reallocation* routine, executed after each thread state change, takes a toll.



(a) nRF52840 (Cortex-M4) (b) ESP32-C3 (RISC-V)

Fig. 3: Overhead of the multicore scheduling feature, measured on single-core hardware.



(a) RP2040/RP2350 (blue/purple) (b) ESP32-S3 (dual Xtensa LX7)

Fig. 4: Multiplication of $N \times N$ matrices, $N \in \{10, 20, ..., 80\}$

The benchmark largely consists of scheduler operations. Thus, mutual exclusion in the scheduler forces mostly sequential execution on the RP2040. In contrast, on the ESP32-S3, parallelization happens differently in hardware, forcing sequential execution less often. Furthermore, one benchmark iteration on ESP32-S3 requires many more ticks than on the RP2040. Thus, ESP32-S3 reaps more benefits from parallelization in this benchmark. In the following, we focus on the *dynamic thread selection* variant for our multicore scheduling.

B. Overhead of Multicore Scheduling

We next measure the overhead of our multicore scheduling feature, compared to the single-core scheduler we started from in section IV. For this, we focus on thread preemption, because, compared to single-core, the multicore feature adds an additional step: inserting the preempted thread back into the runqueue. Figure 3 reports on micro-benchmarks performed on different single-core boards, in which a lower-priority thread sets the flag for a higher-priority thread, resulting in a context switch. We observe that the overhead when the multicore feature is enabled remains quite small, approx. 9.6% on the nRF52840 and 5.3% on the ESP32-C3. Conveniently, we can therefore enable multicore by default in Ariel OS.

C. Speedup of Computation Intensive Tasks

We next benchmark $N \times N$ matrix multiplication, with and without the multicore feature. The computation is split into 2 tasks, where the 1st and 2nd halves of the first matrix are each multiplied with the second matrix. In the single-core configuration, the two computations are executed sequentially. For multicore, the tasks are distributed to two threads that are

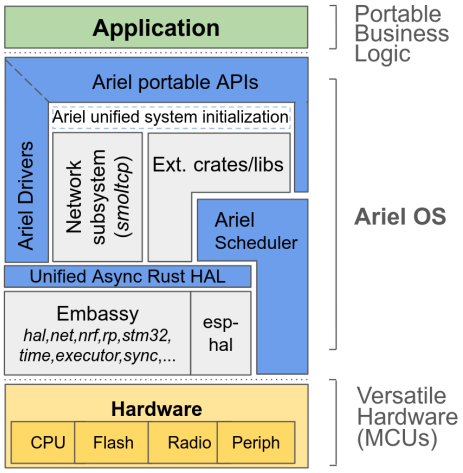


Fig. 5: Ariel OS architecture diagram.

scheduled in parallel. We observe in Fig. 4 that when N is small, IPC overhead dominates parallelization gains. When N grows larger, we observe a performance gain tending towards 2x. We also notice that performance gains are not strictly linear. This is subject for further investigation — it might be due to MCU-specific memory subsystem characteristics. For example, on the RP2040, both cores compete for a shared memory bus [19].

Summary — By using a global scheduling scheme and priorities, we achieve *work-conservation* and *real-time* properties. Our hardware abstraction provides *portability*, and we demonstrated *transparency* via our experimental measurements. We have thus achieved the goals we set in Section III.

VIII. Ariel Operating System Overview

Ariel OS is fully open-source [20]. For basic hardware abstraction and async Rust programming, Ariel OS builds on top of Embassy [8]. Fig. 5 shows how Ariel OS components (in blue) harness the ecosystem of embedded Rust (in grey). In particular, Ariel OS combines the following elements:

Versatile network stack configurations — Ariel OS integrates a network stack (*embassy-net/smoltcp* [21]) combined with additional modules we provide allowing various network configurations. These configuration options include IPv4 and IPv6, HTTP/TCP and CoAP/UDP, over wireless and wired link layers, secured by open standards including COSE, OSCORE, EDHOC, TLS [22] and a curated set of libraries providing cryptographic backends.

Abstracted system initialization — On MCUs, code initializing the system can be very challenging for developers. Ariel OS thus abstracts initialization, e.g., setting up (i) the network stack, (ii) cryptographic material and identities, (iii) the random number generator, (iv) USB peripherals, etc. Configuration is handled at build system level. Convenient defaults are provisioned. Boilerplate is thus minimized and high-level building blocks are provided to application logic.

Unified peripheral APIs — Ariel OS crafted peripheral initialization and setup (for GPIO, I2C, SPI accessing sensors/actuators) to be identical across MCU families. Thereby, application code written once can be compiled for all devices that Ariel OS supports. This is a substantial improvement over the state of the art in embedded Rust (crates such as *embedded-hal* or *embedded-hal-async*) which leave out initialization, and thus lead to a jungle of initialization APIs — and very limited application code portability.

Meta build system — The Ariel OS toolchain takes full advantage of the embedded Rust *crates* ecosystem and the Rust build system *Cargo*. To work around limitations w.r.t. extremely diverse modular target configurations, we wrapped Cargo in *laze* [23], our meta-build system handling the huge matrix of software configurations on various boards.

IX. Conclusions

In this paper we introduced Ariel OS, the first embedded Rust operating system for microcontrollers supporting both single- and multicore preemptive scheduling combined with asynchronous Rust. We assessed experimentally how a unique multicore scheduler can be the convenient default on all supported multicore platforms. Still, applications can opt out of multicore scheduling when parallelization is unneeded. Ariel OS thus enriches the set of available open source tools for secure and efficient distributed computing applications involving sensors/actuators or small networked devices using 32-bit MCUs such as ARM Cortex-M, RISC-V, or ESP32.

References

[1] C. Bormann, M. Ersue, and A. Keränen, "Terminology for Constrained-Node Networks," RFC 7228, May 2014.
[2] O. Hahm *et al.*, "Operating systems for low-end devices in the Internet of Things: a survey," *IEEE Internet of Things Journal*, 2015.
[3] A. Levy *et al.*, "Ownership is theft: Experiences building an embedded OS in Rust," in *ACM PLOS*, 2015.
[4] The Rust Community, "The Embedded Rust Book," https://docs.rust-embedded.org/book/, 2024.
[5] "Hubris OS," https://github.com/oxidecomputer/hubris, 2025.
[6] "Drone OS," https://github.com/drone-os, 2025.
[7] "RTIC," https://github.com/rtic-rs/rtic, 2025.
[8] "Embassy Framework," https://github.com/embassy-rs/embassy, 2025.
[9] "Eclipse ThreadX," https://github.com/eclipse-threadx, 2025.
[10] "FreeRTOS," https://www.freertos.org/, 2025.
[11] "Zephyr Project," https://zephyrproject.org/, 2025.
[12] "Apache NuttX," https://nuttx.apache.org/, 2025, [Online].
[13] E. Baccelli *et al.*, "RIOT: An open source operating system for low-end embedded devices in the IoT," *IEEE Internet of Things Journal*, 2018.
[14] R. I. Davis and A. Burns, "A survey of hard real-time scheduling for multiprocessor systems," *ACM Comput. Surv.*, 2011.
[15] B. B. Brandenburg, "Scheduling and Locking in Multiprocessor Real-Time Operating Systems," in *PhD Diss., Uni. North Carolina*, 2011.
[16] O. Zapata and P. M. Álvarez, "EDF and RM Multiprocessor Scheduling Algorithms : Survey and Performance Evaluation," 2005.
[17] S. Klabnik and C. Nichols, *The Rust Programming Language*, 2022, book chapter 10.2 Traits: Defining Shared Behavior.
[18] "Ariel OS Benchmarks," https://github.com/elenaf9/ariel-os-benchmarks, 2025.
[19] P. Warden, "Understanding the Raspberry Pi Pico's Memory Layout," Online: https://petewarden.com/2024/01/16/understanding-the-raspberry-pi-picos-memory-layout/, January 2024.
[20] "Ariel OS," https://github.com/ariel-os/ariel-os, 2025.
[21] "SmolTCP Net. Stack," https://github.com/smoltcp-rs/smoltcp, 2025.
[22] H. Tschofenig and E. Baccelli, "Cyberphysical Security for the Masses: A Survey of the Internet Protocol Suite for Internet of Things Security," *IEEE Security & Privacy*, 2019.
[23] "Laze Meta-Build System," https://github.com/kaspar030/laze, 2025.